

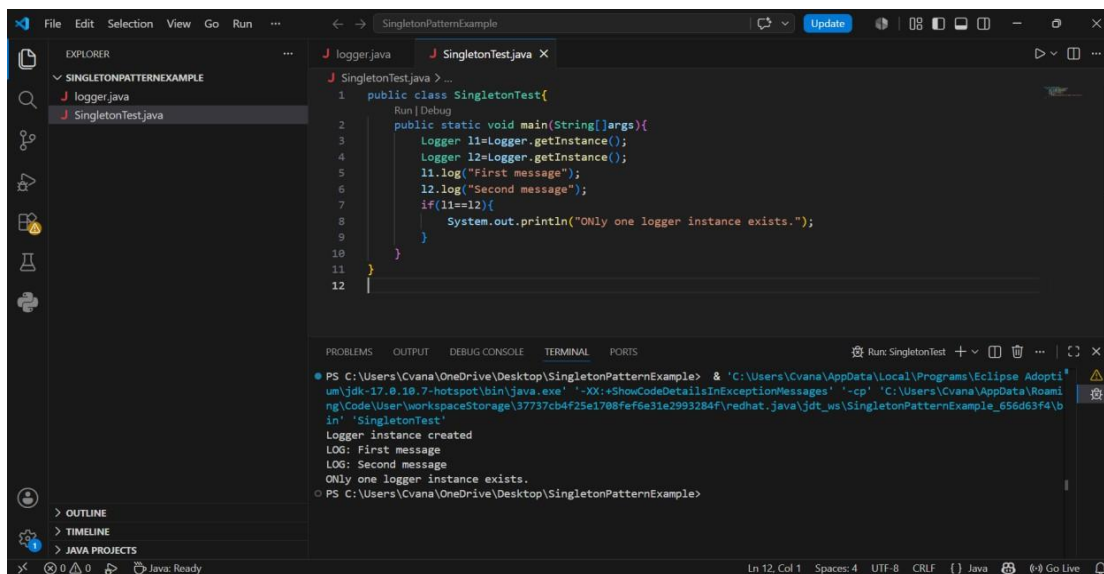
WEEK-1

P.Srikar

Design Patterns And Principles

Exercise 1: Implementing the Singleton Pattern

Output:



```
1 public class SingletonTest{
2     public static void main(String[] args){
3         Logger l1=Logger.getInstance();
4         Logger l2=Logger.getInstance();
5         l1.log("First message");
6         l2.log("Second message");
7         if(l1==l2){
8             System.out.println("Only one logger instance exists.");
9         }
10    }
11 }
12 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Run: SingletonTest

```
PS C:\Users\Cvana\OneDrive\Desktop\SingletonPatternExample> & 'C:\Users\Cvana\AppData\Local\Programs\Eclipse Adopti
um\jdk-17.0.10.7-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Cvana\AppData\Roami
ng\Code\User\workspaceStorage\37737cb4f25e1708fef6e31e2993284f\redhat.java\jdt_ws\SingletonPatternExample_656d63f4\b
in' 'SingletonTest'
Logger instance created
LOG: First message
LOG: Second message
Only one logger instance exists.
PS C:\Users\Cvana\OneDrive\Desktop\SingletonPatternExample>
```

Exercise 2: Implementing the Factory Method Pattern

Output:

```

J Main.java > ExcelFactory
1
2 interface Document {
3     void open();
4 }
5 class WordDocument implements Document {
6     @Override
7     public void open() {
8         System.out.println(x: "Opening Word Document...");
9     }
10 }
11 class PdfDocument implements Document {
12     @Override
13     public void open() {
14         System.out.println(x: "Opening PDF Document...");
15     }
16 }
17 class ExcelDocument implements Document {
18     @Override
19     public void open() {
20         System.out.println(x: "Opening Excel Document...");
21     }
22 }
23 abstract class DocumentFactory {
24     public abstract Document createDocument();
25 }
26 class WordFactory extends DocumentFactory {
27     @Override
28     public Document createDocument() {
29         return new WordDocument();
30     }
31 }
32
33 class PdfFactory extends DocumentFactory {
34     @Override
35     public Document createDocument() {
36         return new PdfDocument();
37     }
38 }
39 class ExcelFactory extends DocumentFactory {
40     @Override
41     public Document createDocument() {
42         return new ExcelDocument();
43     }
44 }
45 public class Main {

```

```

public class Main {
    Run | Debug
    public static void main(String[] args) {
        DocumentFactory f1 = new WordFactory();
        Document d1 = f1.createDocument();
        d1.open();

        DocumentFactory f2 = new PdfFactory();
        Document d2 = f2.createDocument();
        d2.open();

        DocumentFactory f3 = new ExcelFactory();
        Document d3 = f3.createDocument();
        d3.open();
    }
}

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Cvana\OneDrive\Desktop\FactoryMethodPatternExample> javac Main.java
PS C:\Users\Cvana\OneDrive\Desktop\FactoryMethodPatternExample> java Main
Opening Word Document
Opening PDF Document
Opening Excel Document
PS C:\Users\Cvana\OneDrive\Desktop\FactoryMethodPatternExample>

```

Data Structures And Algorithms

Exercise 2: E-commerce Platform Search Function

Output:

Product.java:

```
public class product {  
    int productId;  
    String productName;  
    String category;  
  
    public product(int productId, String productName, String category) {  
        this.productId = productId;  
        this.productName = productName;  
        this.category = category;  
    }  
  
    @Override  
    public String toString() {  
        return "ID: " + productId +  
            ", Name: " + productName +  
            ", Category: " + category;  
    }  
}
```

Main.java:

```
J Main.java > Main > main(String[])  
13: public class Main {  
43:     public static void main(String[] args) {  
53:         int searchId = 104;  
54:  
55:         Product p1 = linearSearch(products, searchId);  
56:  
57:         if (p1 != null)  
58:             System.out.println("Linear Search Found: " + p1.productName);  
59:  
60:         Product p2 = binarySearch(products, searchId);  
61:  
62:         if (p2 != null)  
63:             System.out.println("Binary Search Found: " + p2.productName);  
64:     }  
65: }
```

Op:

```
● PS C:\Users\pbsrao\OneDrive\Desktop\cogDS\week1> javac search.java  
● PS C:\Users\pbsrao\OneDrive\Desktop\cogDS\week1> java search  
Linear Search Found: Watch  
Binary Search Found: Watch  
● PS C:\Users\pbsrao\OneDrive\Desktop\cogDS\week1> █
```

Exercise 7: Financial Forecasting

Output:

```
J Main.java > Main
1 public class Main {
2     static double forecast(double value, double rate, int years) {
3         if (years == 0) {
4             return value;
5         }
6         return forecast(value * (1 + rate), rate, years - 1);
7     }
8
9     Run | Debug
10    public static void main(String[] args) {
11        double currentValue = 1000;
12        double growthRate = 0.10;
13        int years = 3;
14
15        double futureValue = forecast(currentValue, growthRate, years);
16        System.out.println("Future Value = " + futureValue);
17    }
18 }
```

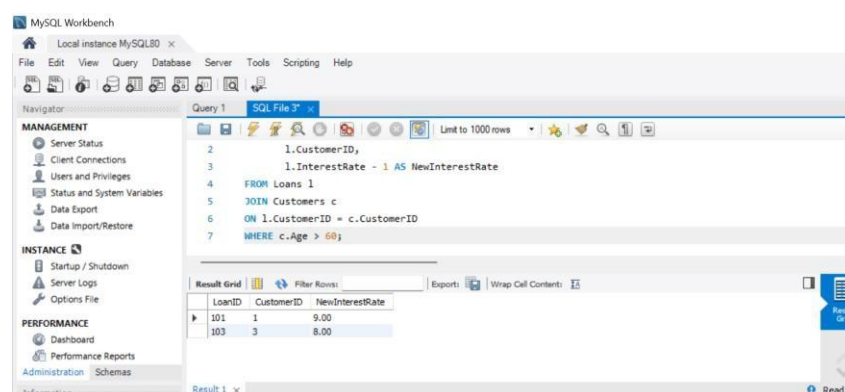
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\pbsra\OneDrive\Desktop\cogDS\week1> javac Main.java
- PS C:\Users\pbsra\OneDrive\Desktop\cogDS\week1> java Main
Future Value = 1331.0
- PS C:\Users\pbsra\OneDrive\Desktop\cogDS\week1>

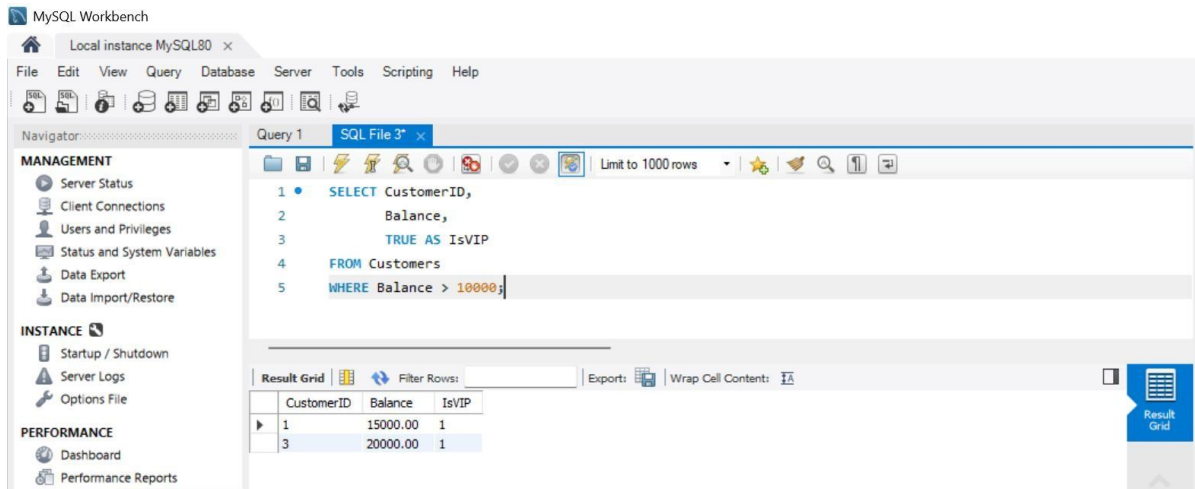
SQL

Exercise 1: Control Structures

Scenario1:



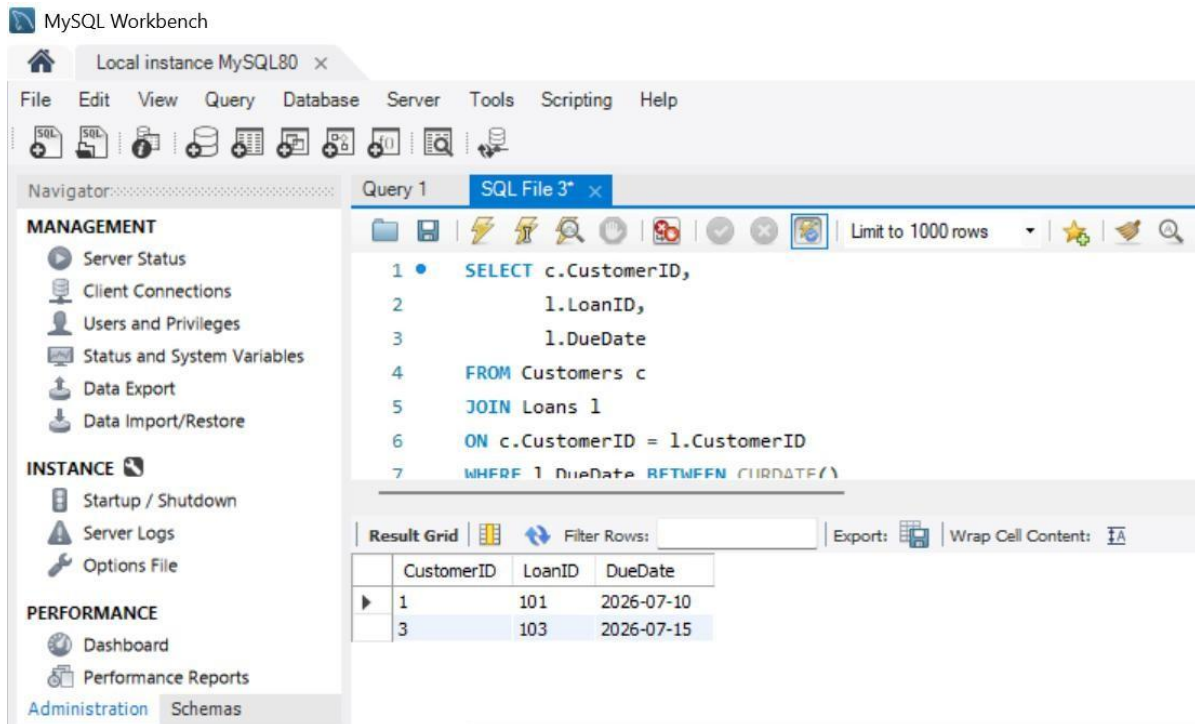
Scenario 2:



A screenshot of the MySQL Workbench interface. The left sidebar shows the 'MANAGEMENT' section with options like Server Status, Client Connections, Users and Privileges, Status and System Variables, Data Export, and Data Import/Restore. The 'INSTANCE' section includes Startup / Shutdown, Server Logs, and Options File. The 'PERFORMANCE' section includes Dashboard and Performance Reports. The main query editor shows a SQL query: `SELECT CustomerID, Balance, TRUE AS IsVIP FROM Customers WHERE Balance > 10000;`. The 'Result Grid' at the bottom displays the results of the query.

CustomerID	Balance	IsVIP
1	15000.00	1
3	20000.00	1

Scenario 3:

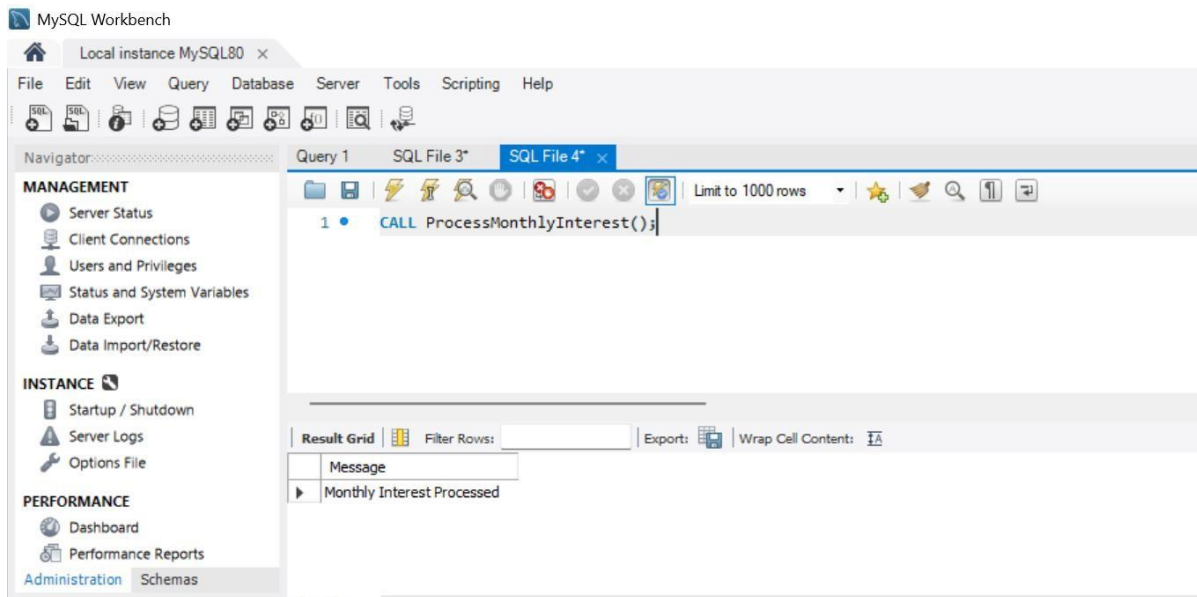


A screenshot of the MySQL Workbench interface. The left sidebar shows the 'MANAGEMENT' section with options like Server Status, Client Connections, Users and Privileges, Status and System Variables, Data Export, and Data Import/Restore. The 'INSTANCE' section includes Startup / Shutdown, Server Logs, and Options File. The 'PERFORMANCE' section includes Dashboard and Performance Reports. The main query editor shows a SQL query: `SELECT c.CustomerID, l.LoanID, l.DueDate FROM Customers c JOIN Loans l ON c.CustomerID = l.CustomerID WHERE l.DueDate BETWEEN CURDATE()`. The 'Result Grid' at the bottom displays the results of the query.

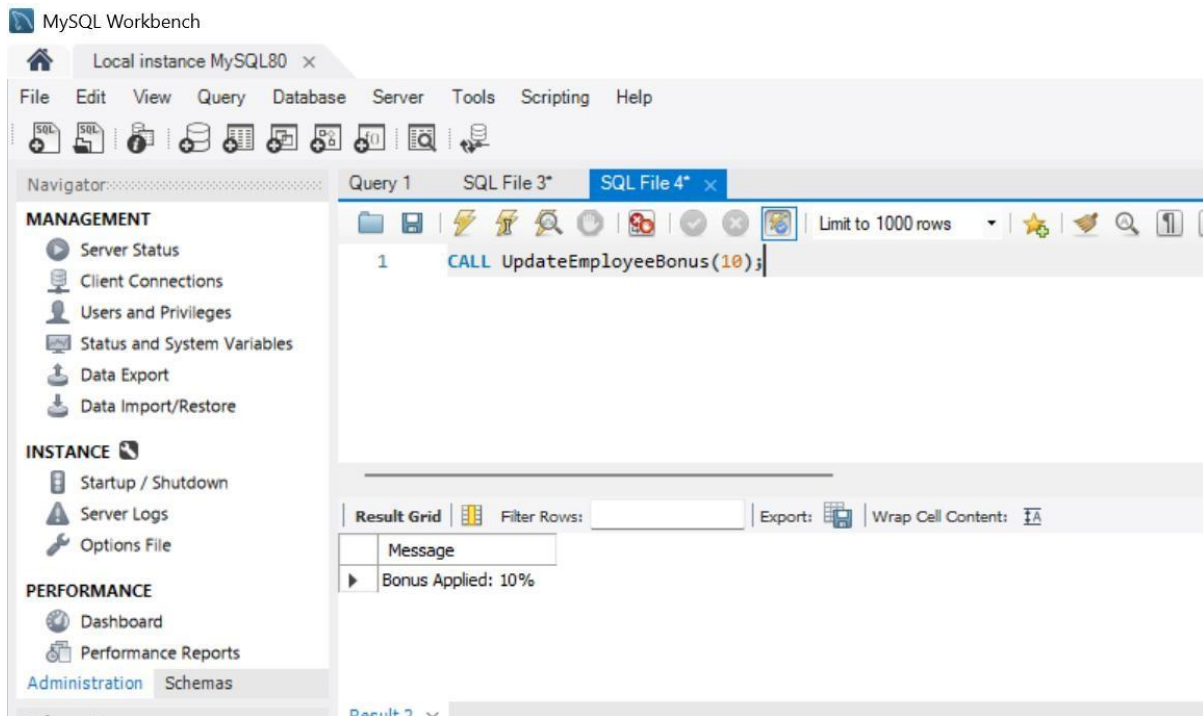
CustomerID	LoanID	DueDate
1	101	2026-07-10
3	103	2026-07-15

Exercise 3: Stored Procedures

Scenario 1:



Scenario 2:



Scenario 3:

